

Rayyan Salman.

Engineering portfolio

I work across CAD and physical hardware. My projects range from controlling a collaborative robot to packaging sensors and processing atmospheric research data.

Georgia Institute of Technology / Expected graduation: May 2028

SELECTED PROJECTS

01 Robot motion & machine vision

LATTICEPT | Mechanical Engineer Intern

Python motion control, calibration, and weld-profile analysis.

02 Search-and-rescue ant robot

Georgia Tech | Team Lead, Mechanical Design

Sensor-head CAD, working drawings, and assembly instructions.

03 High-altitude balloon payload

Georgia Tech VIP | Undergraduate Researcher

EFM and battery enclosures, CAN-FD, and a Python log parser.

ADDITIONAL WORK

FIRST Robotics Competition

2022-2024. Modeled drivetrain and intake assemblies in SolidWorks and supported fabrication through CNC machining and 3D printing.

Adaptive ride-on vehicle

2023. Redesigned seating, controls, and structural supports to adapt a Power Wheels vehicle for a child with cerebral palsy.

rayyansalman08@gmail.com

[LinkedIn](#)

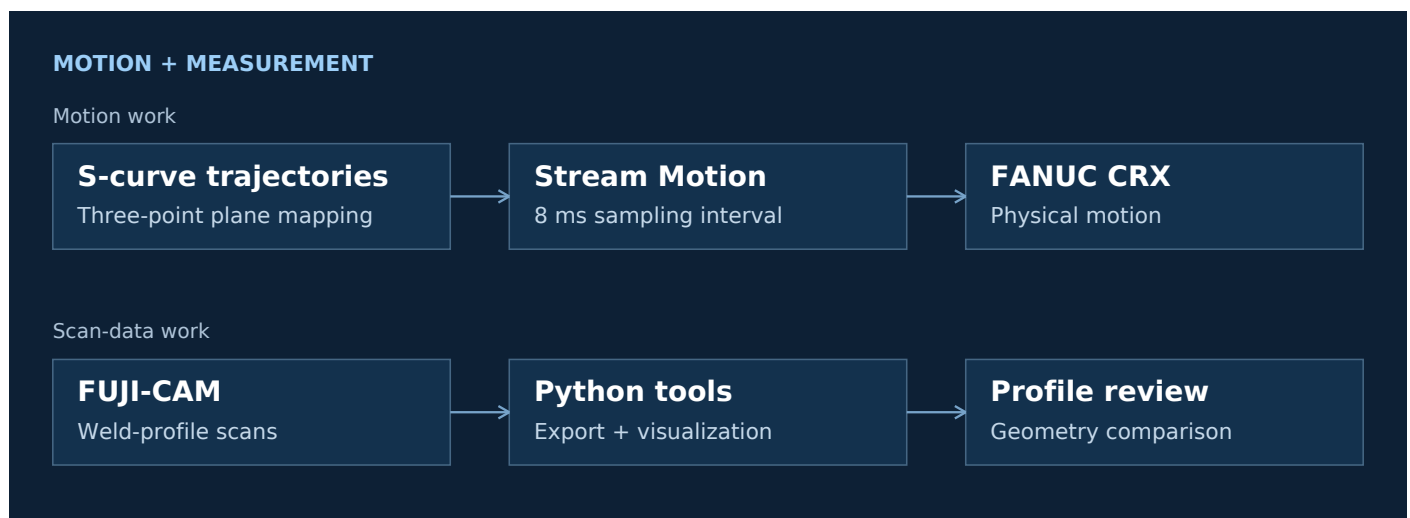
[Online portfolio](#)

01 Robot motion & machine vision

LATTICEPT | Mechanical Engineer Intern

January-July 2026 | Roswell, Georgia

I developed a Python-driven motion workflow for a FANUC CRX-10iA/L and worked with a weld-scanning camera, connecting generated paths and sensor data to practical hardware testing.



Workflow schematic: motion control and scan-data analysis are separate workstreams.

MY CONTRIBUTION

- Built three-point plane transforms and S-curve motion profiles with velocity, acceleration, and jerk limits.
- Implemented UDP command encoding and robot-status parsing for Stream Motion (J519).
- Added offline trajectory analysis and CSV export at a configured 8 ms sampling interval.
- Investigated robot motion faults and built FUJI-CAM profile-analysis tools.

RESULT

A working path-execution workflow for tasks including robotic writing, plus tools for inspecting scan geometry.

The controller separates geometry, motion planning, and communication. It plans straight segments with stops at each waypoint; offline checks support trajectory review before hardware testing.

Python / NumPy / Coordinate transforms / S-curve planning / J519

Calibrated coordinates

FANUC Stream Motion J519 | Python | NumPy

The controller uses a coordinate frame defined by three calibration points. These transforms map a drawing onto that plane and project 3D points back into local coordinates.

COORDINATE TRANSFORM

Three calibration points establish an origin and two perpendicular unit axes. This excerpt maps a local point on that plane into the robot's 3D coordinate system.

```
def to_global(self, u: float, v: float) -> np.ndarray:
    return self.origin + u * self.u_axis + v * self.v_axis
```

Code excerpt: geometry.py / PlaneBasis.to_global

PROJECT A 3D POINT BACK ONTO THE PLANE

Subtracting the origin gives a displacement vector. Dot products with the two perpendicular unit axes recover its local coordinates.

```
def to_local(self, point: np.ndarray) -> Tuple[float, float]:
    """Project a 3D point onto the plane, returning (u, v) coordinates."""
    delta = np.asarray(point[:3]) - self.origin
    u = float(np.dot(delta, self.u_axis))
    v = float(np.dot(delta, self.v_axis))
    return (u, v)
```

Code excerpt: geometry.py / PlaneBasis.to_local

The global transform builds a 3D position from local coordinates; the local transform projects a 3D position onto the calibrated plane. Keeping this geometry separate from motion planning makes each step easier to inspect.

Source excerpts; indentation adjusted for display.

Shaping each move

trajectory.py | SCurveProfile | Pose interpolation

A short move may not have enough distance to reach the requested speed. The S-curve implementation accounts for the distance needed to accelerate and decelerate before sampling the resulting path.

FIND A PEAK VELOCITY THAT FITS

The search compares the move length with twice the acceleration distance, accounting for the symmetric deceleration phase.

```
def _find_max_velocity(self, d: float, a: float, j: float) -> float:
    """Binary search for the max velocity that fits in distance d."""
    v_lo, v_hi = 0.0, self.v_max
    for _ in range(64): # plenty of iterations for convergence
        v_mid = 0.5 * (v_lo + v_hi)
        s = self._accel_distance(v_mid, a, j)
        if 2 * s <= d:
            v_lo = v_mid
        else:
            v_hi = v_mid
    return 0.5 * (v_lo + v_hi)
```

Code excerpt: trajectory.py / SCurveProfile._find_max_velocity

TURN SAMPLED DISTANCE INTO POSES

Each sampled distance becomes a fraction of the segment. The code clamps it to the segment bounds and interpolates the pose.

```
samples = profile.sample(dt)
points = []
for s_pos, _, _ in samples:
    fraction = s_pos / dist if dist > 0 else 0.0
    fraction = max(0.0, min(1.0, fraction))
    pose = start + fraction * delta
    points.append(pose)

return points
```

Reading controller feedback

protocol.py | Binary packet parsing | Status flags

The response parser turns incoming bytes into named fields. The streaming client uses the returned ready flag and sequence number when sending the next planned position.

DECODE THE HEADER AND STATUS BYTE

```
# Header
state.packet_type = struct.unpack(">I", data[0:4])[0]
state.version = struct.unpack(">I", data[4:8])[0]
state.seq_no = struct.unpack(">I", data[8:12])[0]

# Status bits
status_byte = struct.unpack(">B", data[12:13])[0]
state.status = RobotStatus(
    sysrdy=(status_byte & 0b0001) > 0,
    accept_cmd=(status_byte & 0b0010) > 0,
    received_cmd=(status_byte & 0b0100) > 0,
    in_motion=(status_byte & 0b1000) > 0,
)
```

Code excerpt: protocol.py / parse_response

WHAT THE FIELDS REPRESENT

Header	Packet type, protocol version, and sequence number are read as big-endian unsigned integers.
Status flags	Bit masks identify system readiness, command acceptance, command receipt, and motion.
Offline review	Dry-run mode generates trajectories without a robot connection and exports timestamped poses to CSV.

The project includes tests for geometry, packet formats, and S-curve behavior. The 8 ms interval is a sampling setting; offline checks do not establish measured robot timing or motion accuracy.

02 Search-and-rescue ant robot

Georgia Tech | Team Lead, Mechanical Design

August-December 2025

Our team developed a search-and-rescue robot concept for confined environments. I designed the modular sensor head, bringing camera, thermal-sensor, neck-joint, and antenna components into one assembly.



My sensor-head CAD from the Fall 2025 report, with internal mounting features visible.

MY CONTRIBUTION

- Designed the head shell, camera housing, modular plate, thermal-sensor plate, neck-joint plate, and sensor mounts.
- Modeled upper and lower antenna components, connectors, and ball-and-socket joints.
- Created part models, the final subassembly, working drawings, and eight-step assembly instructions.

DESIGN INTENT

The head was designed to rotate about the neck joint for 360-degree sensing coverage. Sliding antenna components reduced its profile.

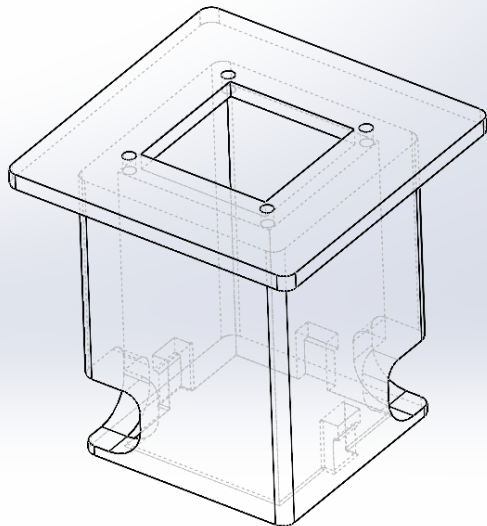
The deliverable was a documented CAD design, not a field-tested rescue system. I also documented an eight-step head assembly sequence.

03 High-altitude balloon payload

Georgia Tech VIP | Undergraduate Researcher

August-December 2025; August 2026-present

My work on the balloon team spans mechanical packaging, embedded communications, and sensor-data processing. I designed the battery case and revised the existing electric-field mill (EFM) enclosure.

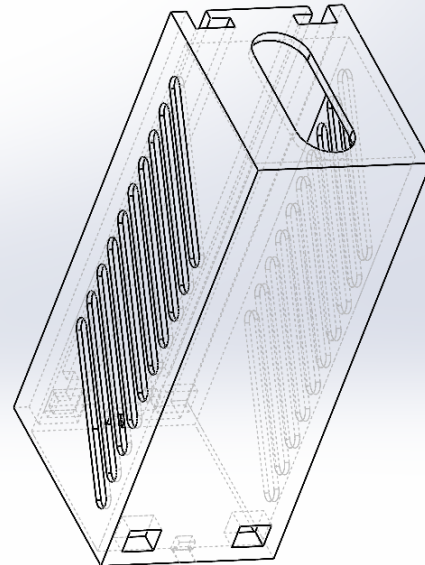


EFM case redesign

Revised enclosure geometry for updated EFM hardware and mounting interfaces.

ASSEMBLY AND SERVICEABILITY

Alongside the CAD work, I researched heat-set inserts and the available installation tools for serviceable enclosure connections.



Battery case design

My enclosure CAD for the balloon payload's battery system.

COMMUNICATIONS AND DATA

I worked on CAN-FD communications using MCP2518 and RP2040 hardware with SocketCAN and PyCyphal. I also wrote the sensor-log parser shown on the next page.

From raw logs to usable data

Python | Regular expressions | CSV / Excel

I wrote a parser to organize more than 1,800 timestamped sensor records for spreadsheet review. It extracts readings from each matching log line and writes a consistent 14-column CSV.

PARSER EXCERPT

```
raw_vals = [x.strip() for x in raw_tuple.split(',')]
val_vals = [x.strip() for x in val_tuple.split(',')]

writer.writerow([
    human_ts.strip(), T.strip(), P.strip(),
    A1.strip(), A2.strip(), UV.strip(),
    *raw_vals, *val_vals
])
```

Excerpt from parse_efm_log.py: split the EFM tuples and write one output row.

OUTPUT FIELDS

CSV columns	Count	Contents
timestamp	1	Human-readable timestamp
T, P	2	Temperature and pressure
A_x, A_y	2	Two A-channel readings
UV	1	UV reading
EFM_raw_1 ... EFM_raw_4	4	Raw EFM values
EFM_val_1 ... EFM_val_4	4	Converted values already in the log

The parser trims whitespace, retains missing readings recorded as None, and skips nonmatching lines. It exports CSV and keeps the converted EFM values supplied in the log.

[View the full parser](#)